

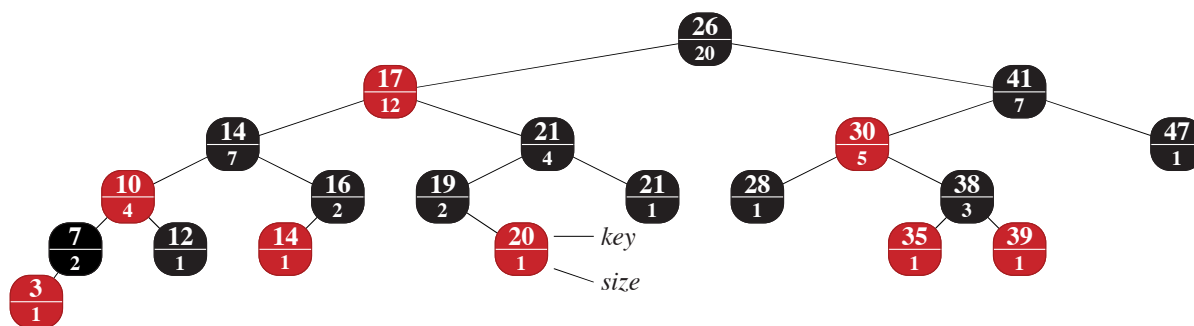


Figure 17.1 An order-statistic tree, which is an augmented red-black tree. In addition to its usual attributes, each node x has an attribute $x.size$, which is the number of nodes, other than the sentinel, in the subtree rooted at x .

Figure 17.1 shows a data structure that can support fast order-statistic operations. An *order-statistic tree* T is simply a red-black tree with additional information stored in each node. Each node x contains the usual red-black tree attributes $x.key$, $x.color$, $x.p$, $x.left$, and $x.right$, along with a new attribute, $x.size$. This attribute contains the number of internal nodes in the subtree rooted at x (including x itself, but not including any sentinels), that is, the size of the subtree. If we define the sentinel's size to be 0—that is, we set $T.nil.size$ to be 0—then we have the identity $x.size = x.left.size + x.right.size + 1$.

Keys need not be distinct in an order-statistic tree. For example, the tree in Figure 17.1 has two keys with value 14 and two keys with value 21. When equal keys are present, the above notion of rank is not well defined. We remove this ambiguity for an order-statistic tree by defining the rank of an element as the position at which it would be printed in an inorder walk of the tree. In Figure 17.1, for example, the key 14 stored in a black node has rank 5, and the key 14 stored in a red node has rank 6.

Retrieving the element with a given rank

Before we show how to maintain the size information during insertion and deletion, let's see how to implement two order-statistic queries that use this additional information. We begin with an operation that retrieves the element with a given rank. The procedure $OS-SELECT(x, i)$ on the following page returns a pointer to the node containing the i th smallest key in the subtree rooted at x . To find the node with the i th smallest key in an order-statistic tree T , call $OS-SELECT(T.root, i)$.

Here is how $OS-SELECT$ works. Line 1 computes r , the rank of node x within the subtree rooted at x . The value of $x.left.size$ is the number of nodes that come

```

OS-SELECT( $x, i$ )
1   $r = x.\text{left.size} + 1$   // rank of  $x$  within the subtree rooted at  $x$ 
2  if  $i == r$ 
3      return  $x$ 
4  elseif  $i < r$ 
5      return OS-SELECT( $x.\text{left}, i$ )
6  else return OS-SELECT( $x.\text{right}, i - r$ )

```

before x in an inorder tree walk of the subtree rooted at x . Thus, $x.\text{left.size} + 1$ is the rank of x within the subtree rooted at x . If $i = r$, then node x is the i th smallest element, and so line 3 returns x . If $i < r$, then the i th smallest element resides in x 's left subtree, and therefore, line 5 recurses on $x.\text{left}$. If $i > r$, then the i th smallest element resides in x 's right subtree. Since the subtree rooted at x contains r elements that come before x 's right subtree in an inorder tree walk, the i th smallest element in the subtree rooted at x is the $(i - r)$ th smallest element in the subtree rooted at $x.\text{right}$. Line 6 determines this element recursively.

As an example of how OS-SELECT operates, consider a search for the 17th smallest element in the order-statistic tree of Figure 17.1. The search starts with x as the root, whose key is 26, and with $i = 17$. Since the size of 26's left subtree is 12, its rank is 13. Thus, the node with rank 17 is the $17 - 13 = 4$ th smallest element in 26's right subtree. In the recursive call, x is the node with key 41, and $i = 4$. Since the size of 41's left subtree is 5, its rank within its subtree is 6. Therefore, the node with rank 4 is the 4th smallest element in 41's left subtree. In the recursive call, x is the node with key 30, and its rank within its subtree is 2. The procedure recurses once again to find the $4 - 2 = 2$ nd smallest element in the subtree rooted at the node with key 38. Its left subtree has size 1, which means it is the second smallest element. Thus, the procedure returns a pointer to the node with key 38.

Because each recursive call goes down one level in the order-statistic tree, the total time for OS-SELECT is at worst proportional to the height of the tree. Since the tree is a red-black tree, its height is $O(\lg n)$, where n is the number of nodes. Thus, the running time of OS-SELECT is $O(\lg n)$ for a dynamic set of n elements.

Determining the rank of an element

Given a pointer to a node x in an order-statistic tree T , the procedure OS-RANK on the facing page returns the position of x in the linear order determined by an inorder tree walk of T .